

Publishing LiDAR Data Topics

1. Experiment Objective

In this experiment, you will learn how to publish the ROS2 standard `sensor_msgs/msg/LaserScan` LiDAR scan data through micro-ROS. Through this example, you will master receiving and parsing UART (serial) LiDAR data, the use of a Ring Buffer, and the micro-ROS publishing configuration for large message types. This example publishes the topic `/scan` at 10 Hz, using the TminiPlus series LiDAR.

2. Experiment Principle

1. Keyword Explanation

Concept	Description
LiDAR	Light Detection And Ranging, a sensor that calculates the distance to a target by emitting laser beams and measuring the reflection time. The TminiPlus is a small, low-cost single-line LiDAR commonly used for robot obstacle avoidance and SLAM
LaserScan message	The ROS2 standard LiDAR scan message type, defined in <code>sensor_msgs/msg/LaserScan</code> , containing fields such as the angle range (<code>angle_min</code> / <code>angle_max</code>), the distance array (<code>ranges</code>) and the intensity array (<code>intensities</code>), describing the result of one 360° scan
ranges array	The core data field of the LaserScan message, storing distance measurements (in meters) in order of increasing angle. The array length is determined by $(angle_max - angle_min) / angle_increment$
Measurement frequency	The number of complete scans completed by the LiDAR per second. The TminiPlus outputs 360° scan data at 10 Hz, with each scan containing multiple angle-distance data points, matching the micro-ROS publish frequency (10 Hz)
Ring Buffer	A fixed-size FIFO data structure that reuses storage space cyclically through head and tail pointers. This example uses a ring buffer to transfer LiDAR serial data between UART interrupt reception and main-loop processing, avoiding data loss
UART serial communication	Universal Asynchronous Receiver/Transmitter. In this example, the TminiPlus LiDAR communicates with the ESP32 through the UART1 interface (TX/RX), requiring the correct baud rate and data format to parse the LiDAR protocol

2. Technical Architecture

This example receives the raw data packets of the TminiPlus LiDAR through the UART1 serial port, and after protocol parsing and ring buffer processing, publishes them as standard `LaserScan` messages at 10 Hz through micro-ROS.

Parameter definition file: `~/esp/microROS_samples/lidar_publisher/main/main.c`

Configuration Item	Value	Description
Node name	tminiplus_publisher	ROS2 node name
Topic name	scan	Topic for publishing LiDAR scan data
Message type	sensor_msgs/msg/LaserScan	ROS2 standard LiDAR message
Publish frequency	10 Hz	Timer period 100 ms
QoS mode	best_effort	Large messages use best-effort transmission to reduce latency
frame_id	laser_frame	LiDAR coordinate frame identifier

Data flow: **TminiPlus LiDAR** → **UART1 interrupt reception** → **ring buffer** → **LiDAR protocol parsing (tminiplus.c)** → **LaserScan message** → **micro-ROS timer callback (fills the timestamp and publishes)**.

The program also has a built-in Agent health check mechanism: when consecutive publish failures reach 10, it proactively pings the Agent to check the connection status.

3. Core Topic Quick Reference

Firmware Uplink (Sensor Data)

Topic	Message Type	QoS	Description
/scan	sensor_msgs/msg/LaserScan	best_effort	LiDAR scan data (360° distance values), published at 10 Hz

Usage Example:

```
# view the LiDAR data (compressed array output)
ros2 topic echo /scan --no-arr
```

4. Middleware Configuration

micro-ROS uses a **static memory model** (`RCUTILS_AVOID_DYNAMIC_ALLOCATION=ON`), all entity memory is pre-allocated at compile time, and runtime dynamic creation is not supported.

Configuration file: `~/esp/extra_components/micro_ros_espidf_component/colcon.meta`

- | `RMW_UXRCE_TRANSPORT` | `udp` | UDP transport (Wi-Fi communication) |
- | `RMW_UXRCE_MAX_PUBLISHERS` | `6` | Maximum number of publishers |
- | `RMW_UXRCE_MAX_SUBSCRIPTIONS` | `6` | Maximum number of subscribers |
- | `RMW_UXRCE_MAX_NODES` | `1` | Maximum number of nodes |
- | `RMW_UXRCE_MAX_SERVICES` | `0` | Services not supported |
- | `RMW_UXRCE_MAX_CLIENTS` | `0` | Clients not supported |
- | `RMW_UXRCE_MAX_HISTORY` | `1` | History depth = 1, no message queuing |

| `UCLIENT_UDP_TRANSPORT_MTU` | `4096` | Maximum UDP packet size 4 KB |
| `UCLIENT_MIN_HEARTBEAT_TIME_INTERVAL` | `1` | Agent heartbeat interval 1 ms |

Key Constraints:

- `MAX_HISTORY=1` means there is no message buffer queue, so uplink topics (firmware → host) uniformly use **best_effort** QoS
- `MAX_SERVICES=0`, `MAX_CLIENTS=0` means only topic communication is supported, not Service/Client
- After modifying `colcon.meta`, the micro-ROS library must be recompiled for the changes to take effect: run `idf.py clean-microros` in the project directory to clean and rebuild the micro-ROS library, then `idf.py build` to compile the firmware

3. Experiment Steps

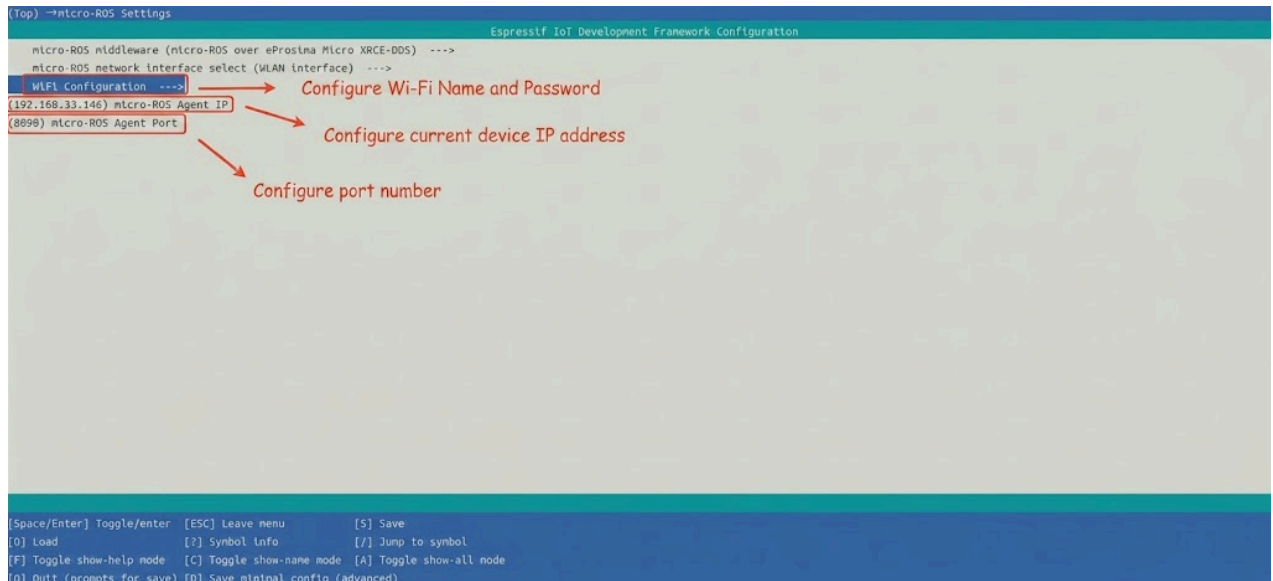
1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✓ Required
USB Type-C cable	✓ Required
Wi-Fi network (2.4 GHz)	✓ Required
TminiPlus LiDAR	✓ Required (connected via UART1)

2. Prerequisites

1. The ESP32-IDF development environment has been set up
2. The MicroROS-V2 control board is connected to the computer via USB
3. Configure the Wi-Fi SSID, password, Agent IP and port via `idf.py menuconfig` (the `micro_ros` menu item)

```
cd ~/esp/microROS_samples/lidar_publisher
source ~/esp/esp-idf/export.sh
idf.py menuconfig
```



4. Select example-app setting to set the domain ID to be consistent with the virtual machine



After configuration, press "s" to save and press "q" to exit the configuration interface.

4. Connect the TminiPlus LiDAR to the UART1 interface of the control board via serial

3. Start micro_ros_agent (Terminal 1)

Start the micro-ROS Agent on the ROS2 host:

```
cd ~/uros_ws
source install/local_setup.bash
ros2 run micro_ros_agent micro_ros_agent udp4 --port 8090 -v4
```

4. Build and Flash (Terminal 2)

```
cd ~/esp/microROS_samples/lidar_publisher
source ~/esp/esp-idf/export.sh
idf.py build flash monitor
```

5. How to Stop

1. Press `Ctrl+]` in Terminal 2 to exit the serial monitor (stop firmware execution)
2. Press `Ctrl+C` in Terminal 1 to stop micro_ros_agent

4. Experiment Result

After the build and flash succeed, on the ROS2 host:

View the LiDAR scan data:

```
ros2 topic echo /scan
```

Expected output (updated at 10 Hz):

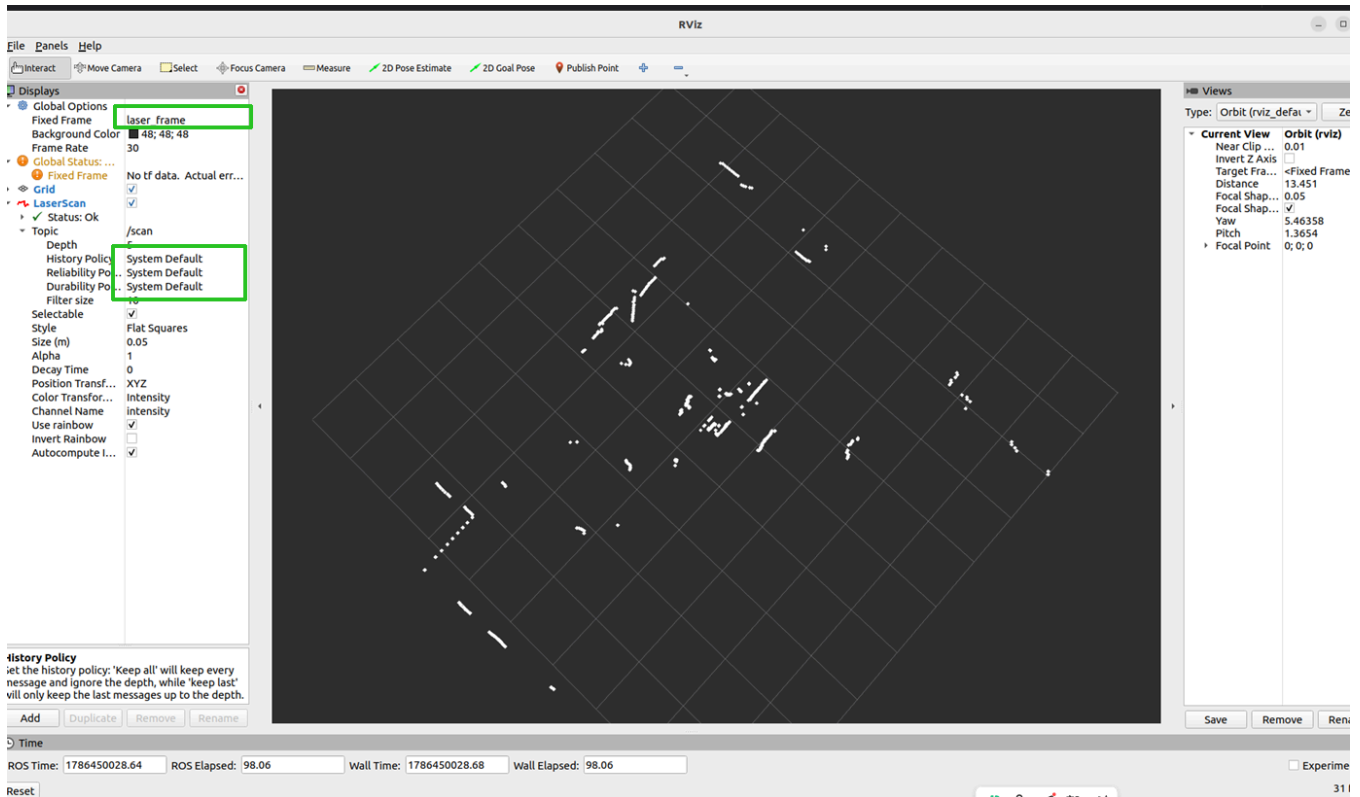
```
header:
  stamp:
    sec: xxx
    nanosec: xxx
  frame_id: laser_frame
angle_min: 0.0
angle_max: 6.283185 # 2π, 360 degrees
angle_increment: ...
range_min: 0.05
range_max: 12.0
ranges: [0.523, 0.518, 0.521, ...]
intensities: [...]
```

Visualize the LiDAR data in RViz:

```
rviz2
```

In RViz:

1. Add `/scan` via By topic → LaserScan display
2. Set the Fixed Frame to `laser_frame`
3. The point cloud image of the LiDAR scan can then be seen
4. Change all topic quality settings to system default



The control board serial port will output:

```
I (xxx) MAIN: Connecting agent: 192.168.x.x:8888
I (xxx) MAIN: Connected agent: 192.168.x.x:8888
I (xxx) MAIN: time synced, offset=xxx ms
I (xxx) MAIN: wi-Fi power save disabled
```

5. Program Analysis

Source code paths:

- Firmware source code: `~/esp/microROS_samples/lidar_publisher/main/main.c`
- LiDAR driver:
`~/esp/microROS_samples/lidar_publisher/components/lidar_tminiplus/lidar_tminiplus.h`,
`tminiplus.c`
- UART driver: `~/esp/microROS_samples/lidar_publisher/components/uart/uart1.c`

1. Main Program Logic

`main/main.c` is the program entry point, and the flow is as follows:

1. In `app_main()`:
 - Calls `uart1_init()` to initialize UART1 serial communication

- Calls `tminiplus_GPIO_Init()` to initialize the LiDAR control GPIO (motor enable, etc.)
- Sends the start command `{0xA5, 0x60}` to start the LiDAR scan
- Delays 1 second to wait for the LiDAR to start
- Initializes the Wi-Fi network and disables power saving mode
- Sets the `frame_id` of the LaserScan message
- Creates `micro_ros_task`

2. In `micro_ros_task`:

- Uses a **loop-and-retry connection** approach to initialize the support layer (slightly different from the separated ping-initialization in the previous examples)
- Creates the node `tminiplus_publisher`
- Publishes the topic `scan` in best_effort mode, with message type `sensor_msgs/msg/LaserScan`
- Creates a 100 ms period timer (10 Hz) bound to the LiDAR data publishing callback
- In the timer callback, fills the timestamp and then publishes
- Triggers the Agent health check on continuous publish failures
- Performs ROS time synchronization every 30 seconds

2. Key Components

`components/lidar_tminiplus/lidar_tminiplus.h` — LiDAR driver interface:

Function	Description
<code>tminiplus_GPIO_Init()</code>	Initializes the LiDAR control GPIO (motor enable pin)
<code>Lidar_tminiplus_Get_Msg()</code>	Gets the LaserScan message (value copy)
<code>Lidar_tminiplus_Get_Msg_Ptr()</code>	Gets the LaserScan message pointer (zero copy)
<code>Lidar_tminiplus_Set_Frame_Id(const char *frame_id)</code>	Sets the message frame_id
<code>Lidar_tminiplus_stop_Scan(uint8_t on)</code>	Starts/stops the LiDAR scan
<code>Lidar_tminiplus_Get_Distance(uint16_t angle)</code>	Gets the distance value at a specified angle

`components/lidar_tminiplus/tminiplus.c` — LiDAR serial protocol parsing:

Processes the serial data protocol of the TminiPlus LiDAR, receiving raw data packets through UART interrupts, parsing them into angle-distance pairs, and organizing them into a LaserScan message.

`components/ring_buffer/ring_buffer.c` — Ring buffer:

Provides an efficient FIFO data buffer for data transfer between UART interrupt reception and main-loop processing.

`components/uart/uart1.c` — UART1 driver:

Function	Description
<code>Uart1_Init()</code>	Initializes the UART1 peripheral and configures parameters such as the baud rate
<code>Uart1_Send_Data(uint8_t *data, size_t len)</code>	Sends data through UART1

LaserScan message structure:

The pointer returned by `Lidar_tminiplus_Get_Msg_Ptr()` points to a pre-allocated `sensor_msgs__msg__LaserScan` struct. In the timer callback, the timestamp is filled in and the message is published directly, avoiding the memory copy overhead of large messages.

LiDAR start control:

Source file: `~/esp/microROS_samples/lidar_publisher/main/main.c`

```
static const uint8_t s_start_cmd[2] = {0xA5, 0x60};

// Send the start command
Uart1_Send_Data((uint8_t *)s_start_cmd, sizeof(s_start_cmd));
vTaskDelay(pdMS_TO_TICKS(1000)); // wait for the LiDAR to start
```

Timer callback `timer_lidar_callback`:

Source file: `~/esp/microROS_samples/lidar_publisher/main/main.c`

```
static void timer_lidar_callback(rcl_timer_t *timer, int64_t last_call_time)
{
    RCLC_UNUSED(last_call_time);

    if (timer == NULL) {
        return;
    }

    sensor_msgs__msg__LaserScan *msg_lidar = Lidar_tminiplus_Get_Msg_Ptr();
    if (msg_lidar == NULL) {
        return;
    }

    // Fill the timestamp
    struct timespec ts = get_timespec_now();
    msg_lidar->header.stamp.sec = (int32_t)ts.tv_sec;
    msg_lidar->header.stamp.nanosec = (uint32_t)ts.tv_nsec;

    // Publish (zero copy, avoiding the memory copy overhead of large messages)
    rcl_ret_t ret = rcl_publish(&publisher_lidar, msg_lidar, NULL);
    if (ret == RCL_RET_OK) {
        s_publish_fail_count = 0U;
        return;
    }
}
```



```

s_publish_fail_count++;

if ((s_publish_fail_count <= PUBLISH_FAIL_THRESHOLD) ||
    ((s_publish_fail_count % 10U) == 0U)) {
    printf("Failed: rcl_publish(&publisher_lidar, msg_lidar, NULL) | line=%d rc=%d\n",
        __LINE__, (int)ret);
}

// When consecutive failures reach the threshold, trigger the Agent health check
if (s_publish_fail_count >= PUBLISH_FAIL_THRESHOLD) {
    s_need_agent_check = true;
}
}

```

Agent health check mechanism (in the main loop):

Source file: `~/esp/microROS_samples/lidar_publisher/main/main.c`

```

if (s_need_agent_check) {
    rmw_ret_t ping_ret = rmw_uros_ping_agent(AGENT_PING_TIMEOUT_MS, AGENT_PING_ATTEMPTS);
    if (ping_ret == RMW_RET_OK) {
        ESP_LOGI(TAG, "agent ping ok");
    } else {
        ESP_LOGW(TAG, "agent ping failed");
    }
    s_need_agent_check = false;
}
}

```

time offset management:

Unlike the previous examples, this is the first example to use the `s_have_valid_time_offset` flag: if the time synchronization fails but has succeeded before, the previous offset is retained and continues to be used; if it has never succeeded, the local uptime is used as the timestamp.

6. Common Problems

Problem	Cause	Solution
The LiDAR does not start	The start command was not sent or the power supply is insufficient	Check the UART1 connection and confirm the LiDAR power supply is normal (usually requires 5 V)
No data on the <code>/scan</code> topic	Serial data parsing failure or baud rate mismatch	Confirm the LiDAR baud rate matches the UART1 configuration, and check the serial parameters in <code>tminiplus.c</code>
A large number of zeros or abnormal values in the LiDAR data	The LiDAR is blocked or out of the measurement range	Confirm there are no obstacles blocking the LiDAR; the measurement range is 0.05 m ~ 12 m

Problem	Cause	Solution
<code>ros2 topic echo /scan</code> outputs too much data	The LaserScan message contains a large number of distance values	Use the <code>--no-arr</code> parameter to compress the array output: <code>ros2 topic echo /scan --no-arr</code>
Publish failure after the Agent connection drops	UDP connection interrupted	The code has a built-in Agent health check; it will proactively ping the Agent after 10 consecutive failures
Timestamp resets to zero	Time synchronization failed and no historical offset	Ensure the Agent is running normally at startup; the code uses the local uptime as a fallback